The American Express Campus Challenge 2025

Mohammad Umam Ali
Polaki Snehitha
Neeraj Sharma

IIT Guwahati

Skills Matters

Index

SUMMARY

1.

FEATURE ENGINEERING

Advanced Feature Engineering:

We engineered a powerful and diverse feature set by applying advanced NLP to offer text and by analyzing time-based user activity.

Ensuring Model Stability:

A rigorous, multi-stage process, including statistical tests and adversarial validation, was used to identify and remove unstable features, making the model robust against data drift.

Model-Driven Feature Selection:

An XGBRanker model's feature importance scores validated our engineering efforts and guided the final selection of the most predictive features.

Final Outcome:

This entire process resulted in a lean, robust, and high-performing model built on the top 14 most impactful features.

2.

MODEL ANALYSIS

Building a Robust Foundation

We first built a stable base model by using adversarial validation to diagnose and fix severe feature drift that caused initial models to fail, allowing us to build a well-generalized model for all future test cases.

Stacking for Performance:

We then developed a stacking ensemble, using the scores from multiple rankers (XGBoost, LightGBM, CatBoost) as features for a final logistic regression meta-model.

Optimizing the Ensemble

An ablation study showed LightGBM was the dominant model in the stack, which allowed us to remove the less impactful CatBoost to create a leaner, more efficient model.

Final Outcome

This optimized and stable stacked ensemble significantly outperformed any single model, proving our strategy's success.

Feature Cleaning & Selection

Step 1: Feature Cleaning for Robustness

Handle Missing Data: Identified and dropped 26 columns where over 90% of values were missing,
Eliminate Zero Variance: Removed 59 columns exhibiting no variance (standard deviation = 0),
Normalise Skewed Features: Applied log1p to skewed numeric features based on mean-median imbalance
Imputation: Numericals features with mean and categorical features with mode

Step 2: Univariate Numerical & Categorical Drift Detection

A total of 119 features (97 numerical with KS-stat > 0.1, 22 categorical with p-value < 0.05) were removed due to significant distributional drift between training and unseen data.

Step 3: Multivariate Drift Detection with Adversarial Validation

A binary classifier was trained to distinguish training from unseen data, capturing multivariate distribution differences.
An AUC > 0.7 indicated strong drift, confirming the model could reliably separate the two datasets.
As a result, 45+ high-drift features (e.g., f320, f317, f141, f10) were removed to improve model generalisability.

Feature Engineered

From train Data

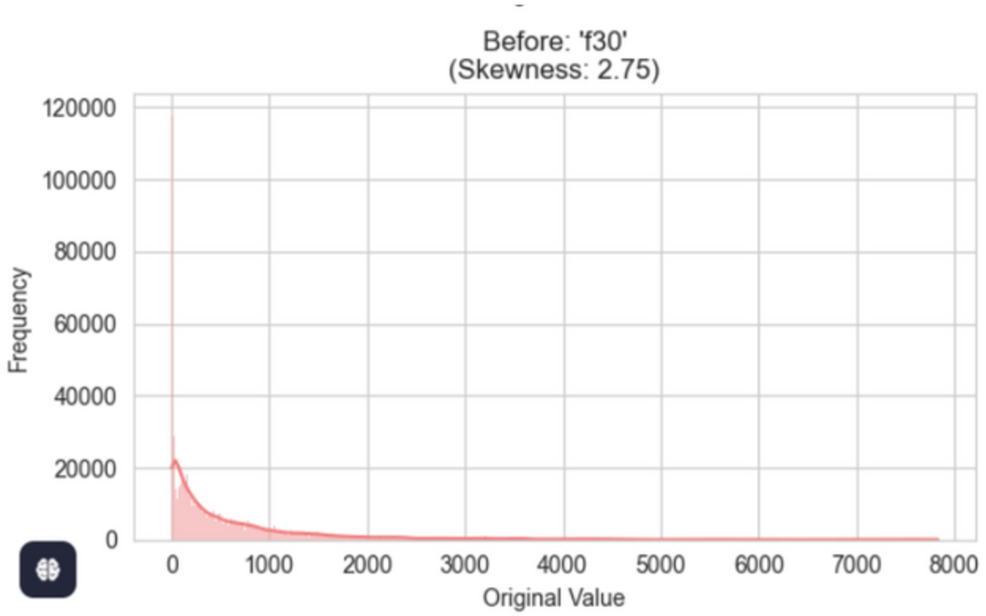
f_total_debit_30d, f_total_debit_180d, time_since_last_event, time_since last_system_event, time_band

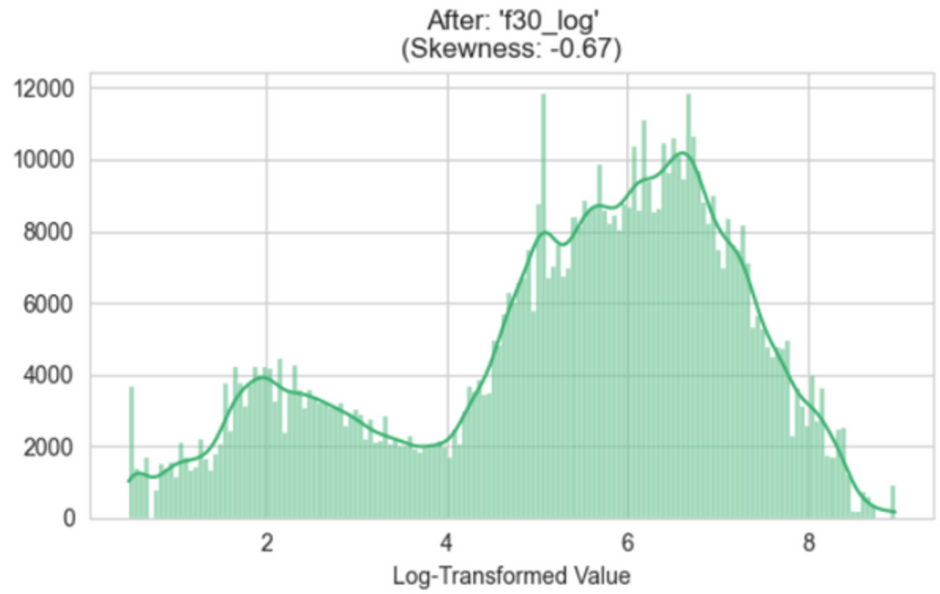
From offers & transaction

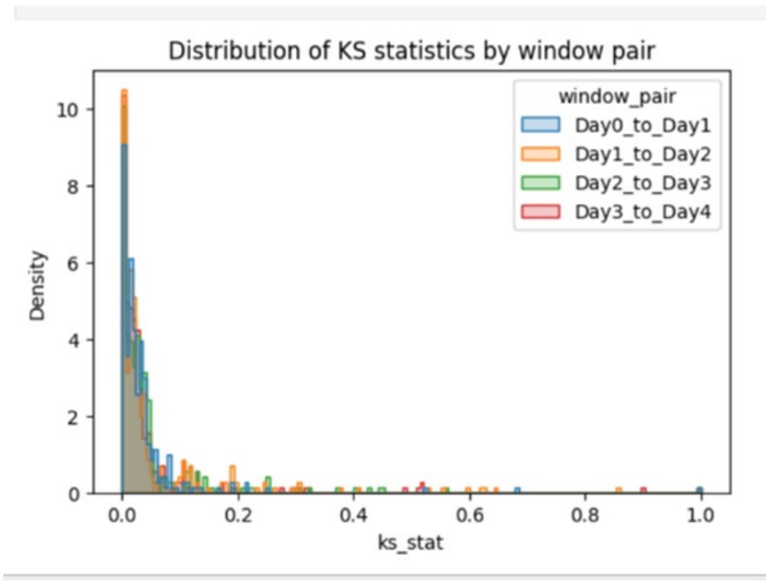
discount_rate, NLP based features using verctor embeddings and dimilarity score for offerings name

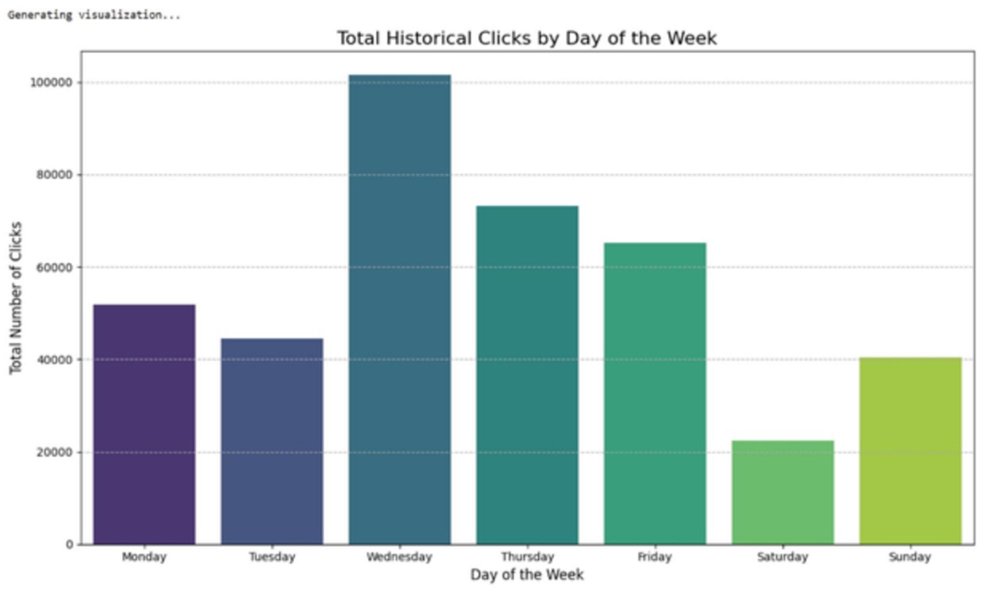
From Events Data

offer_impressions_last_1h, offer_clicks_last_1h, offer_impressions_last_3h, offer_clicks_last_3h, offer_impressions_last_6h, offer_clicks_last_6h, offer_impressions_last_9h, offer_clicks_last_9h, placement_historical_ctr, daywise_ctr









Text Feature Engineering: Transforming Offerings Name using NLP

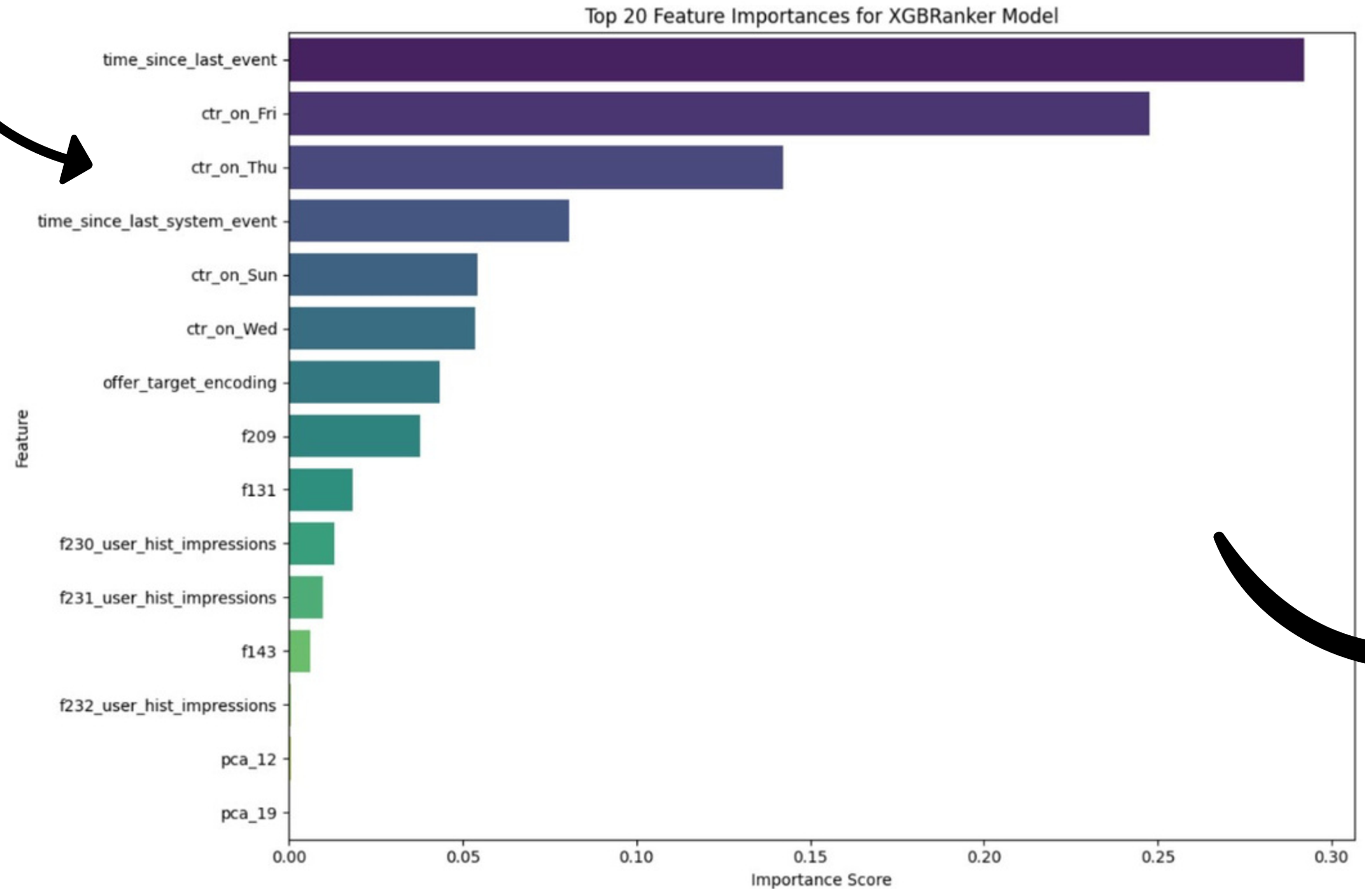
1. Text Pre-processing
To track user behavior, offer descriptions were cleaned and the data was sorted chronologically by session.

2. Text Vectorization
A SentenceTransformer generated 384-dimensional text vectors, which were then efficiently reduced to 32 dimensions using PCA.

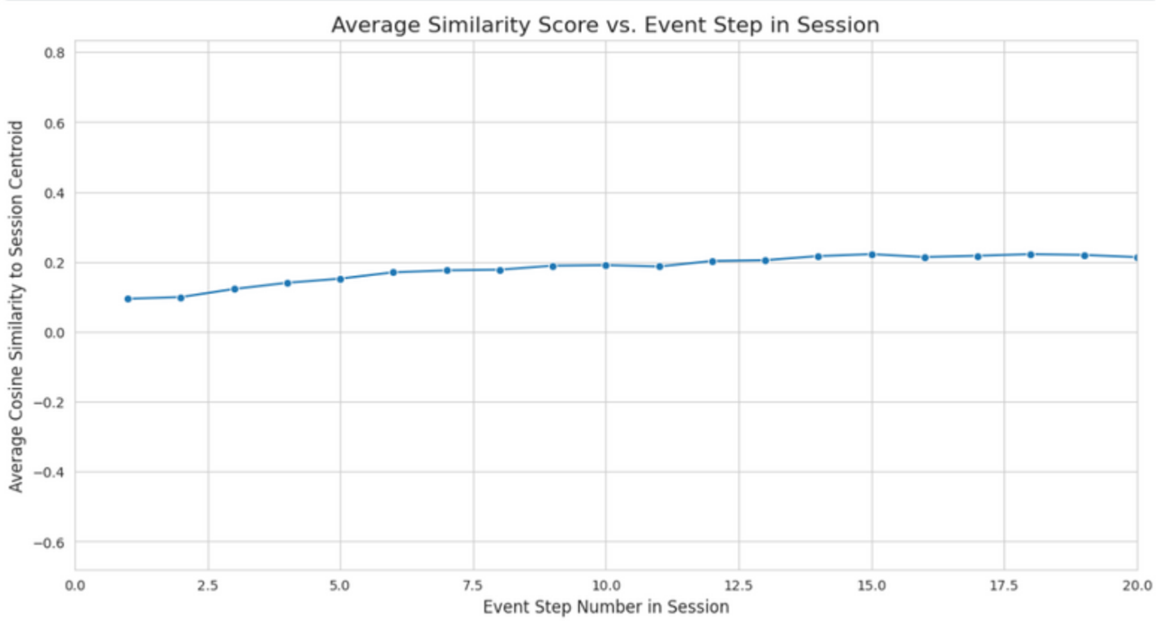
3. PCA Components
The 32 PCA-reduced vectors were unpacked into new columns, with each column representing a component of the offer text's meaning.

4. Session Similarity
To capture immediate user interest, this feature calculates an item's cosine similarity to the user's live session activity.

We applied target encoding to the offer_id, converting each ID into a numerical feature based on its historical performance.



The plot shows a clear upward trend, indicating that as a session progresses, a user's Browse behavior becomes increasingly focused.



Insights from the Feature Importance Graph

- **Temporal Features Dominate:** Time-based features like time_since_last_event and time_since_last_system_event are by far the most powerful predictors in our model.
- **Behavioral Patterns are Crucial:** User click-through rates (CTR), especially on specific days like Friday and Thursday, hold significant predictive power.
- **Engineered Features Validated:** Crucially, our custom-engineered features proved their value. Both offer_target_encoding and text-based pca components appeared in the top features, confirming the success of our feature engineering efforts.

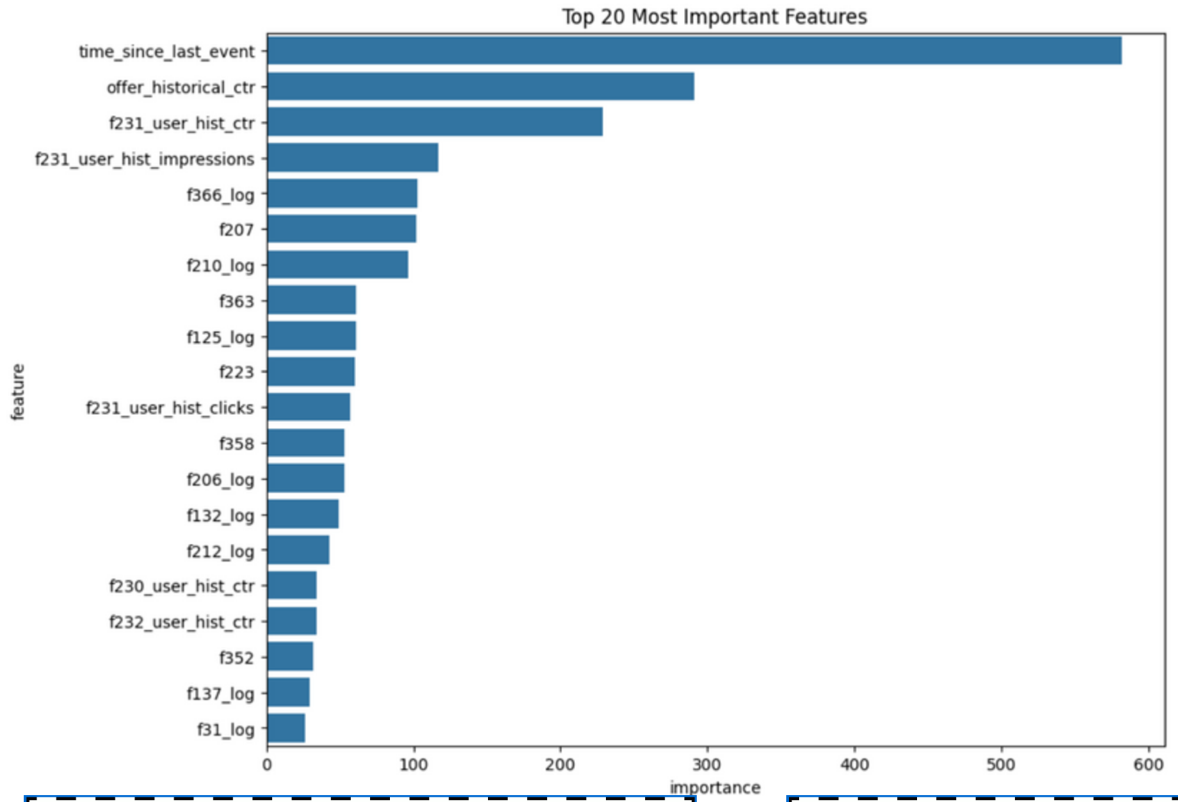
Action Taken & Final Outcome

- **Selective Reduction:** Based on the clear drop-off in importance scores, we performed feature selection by keeping only the top 15 features and discarding the rest.
- **Result:** This leads to a leaner, more focused, and robust model. By removing features with low importance, we reduce noise, decrease the risk of overfitting, and significantly improve training speed.

Model Technique/Algorithm Details

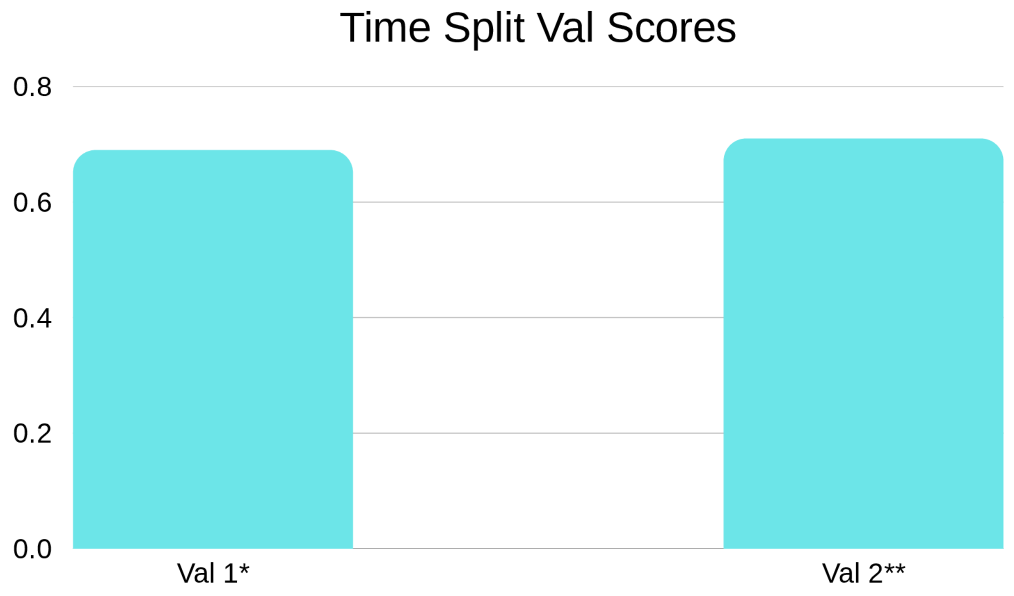
Our initial models, a **powerful XGBoost ranker with advanced, leak-proof historical features**, achieved a very high validation **MAP@7 of ~0.71** on a **time series split, trained on day 0 validate on day 1*, then train on day 0 & 1 and validated on day 2.****

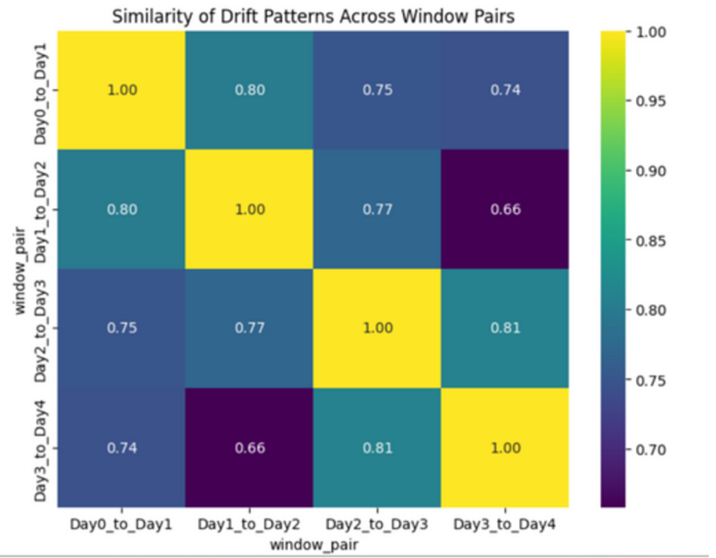
- Direct NDCG optimization via XGBoost's rank:ndcg
- Tree ensembles excel on sparse, high-cardinality tabular data
- Lambda-gradients weight top-rank swaps by ΔNDCG
- Final Ranks transform into probabilities by sigmoid function

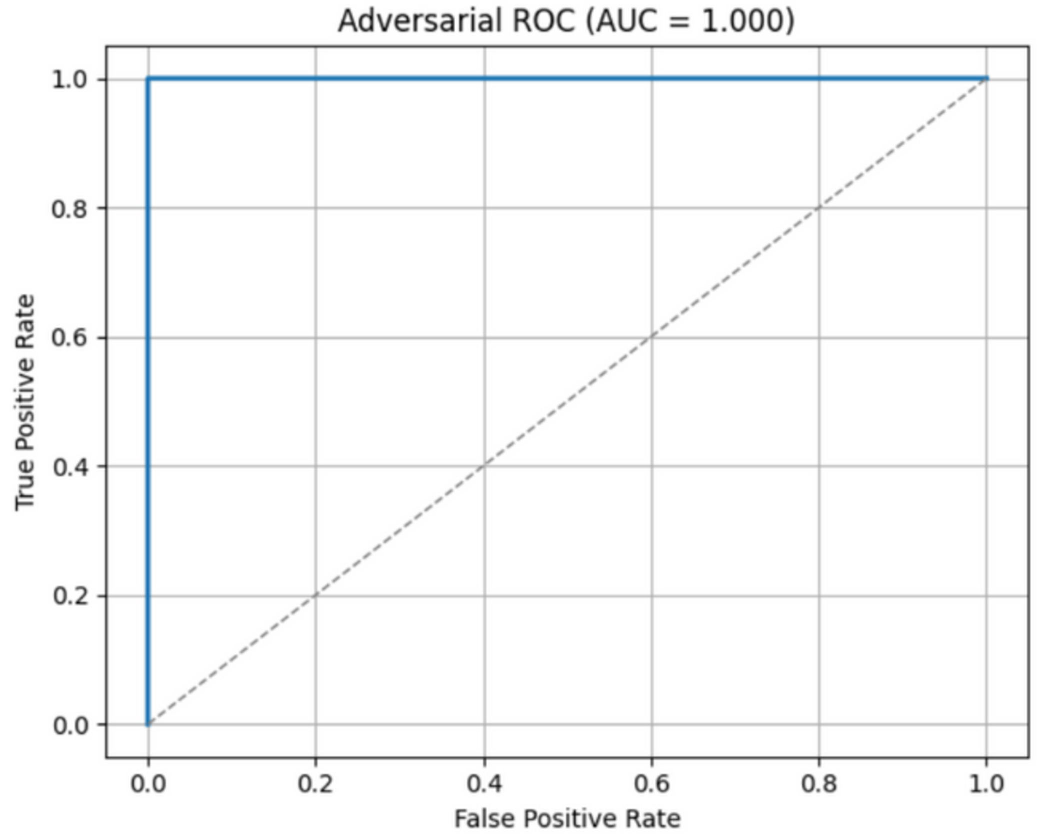


Grouped Labeled Data (queries g1..gM) → XGBoost ranker (predict s_i per item) → Lambda Gradients $\lambda_i \propto |\Delta NDCG_{ij}| * \sigma(-(s_i - s_j))$ → Tree Fitting on λ_i → Updated Scores & NDCG@K

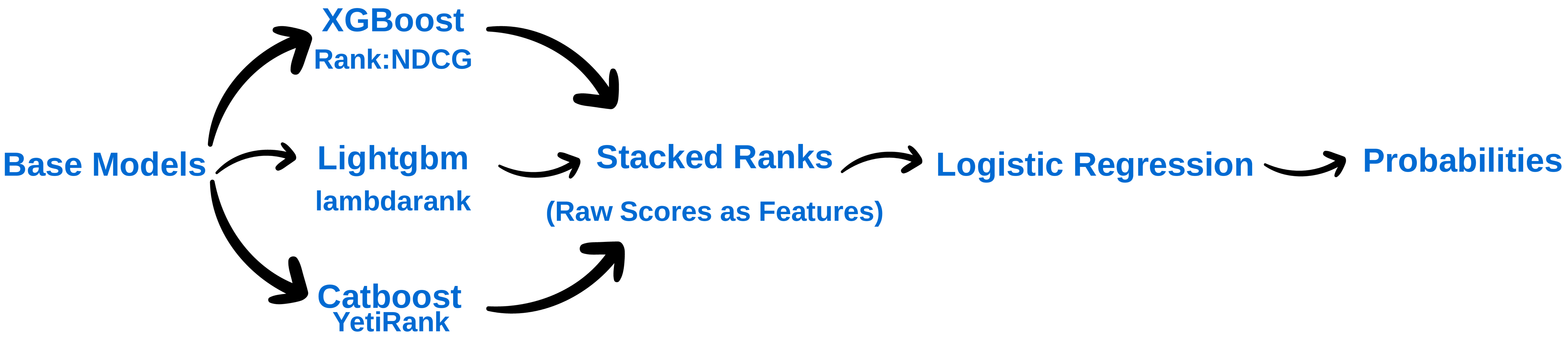
Although our XGBoost ranker achieved MAP@7 ≈ 0.71 on time-series validation, it collapsed to 0.56 on the leaderboard due to feature drift. Adversarial validation with a LightGBM classifier gave AUC = 1.0—confirming that train and test originate from different distributions. By removing high-drift features (using KS and chi-squared tests), we stabilized the adversarial AUC to 0.82 and recovered our MAP@7 to ~0.53, allowing us to make a generalized model.







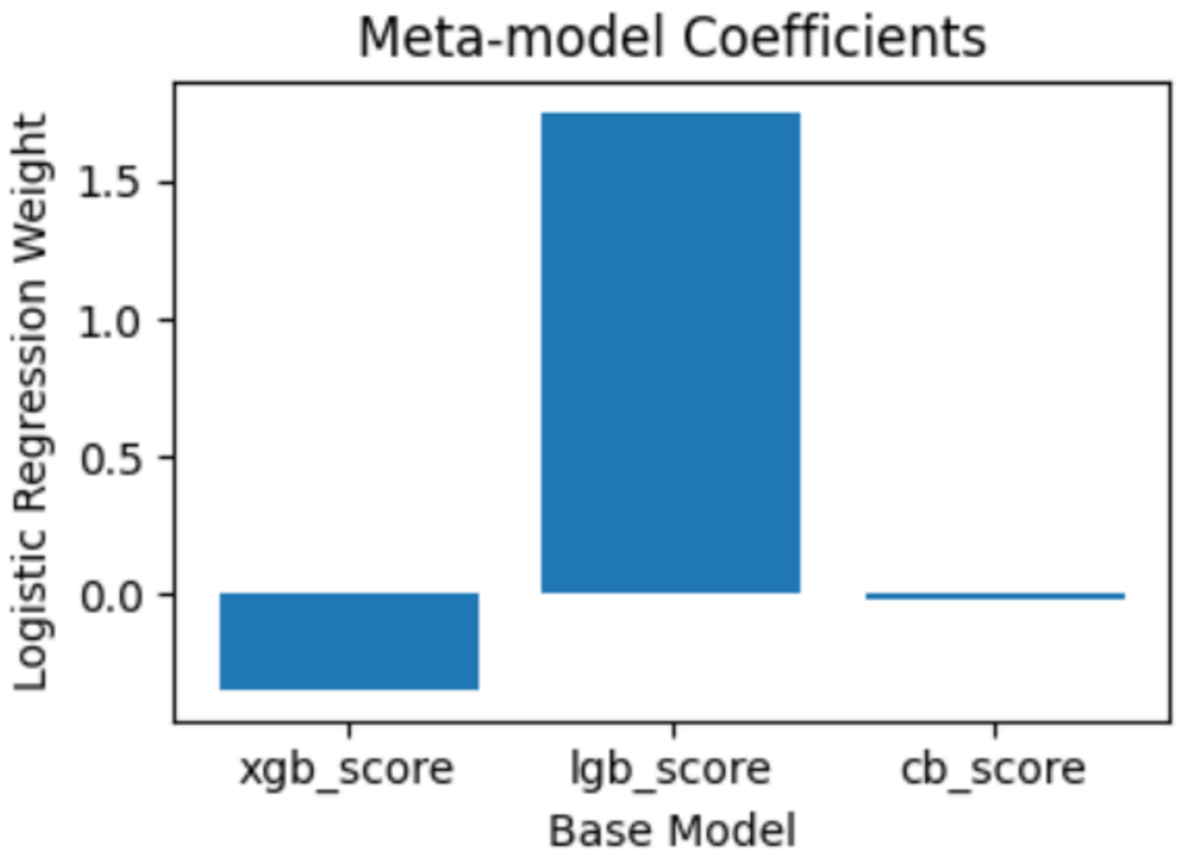
Stacking Ensembling Approach



Stacked Model Analysis

- The ensemble model is more robust to feature drift than any single model, leading to a stable MAP score across various time splits.
- Stacked diverse rankers to reduce variance.
- Raw rank scores used as meta features
- No. of Variable Used:

1. XGboost-14
2. Lightgbm- 14
3. Catboost- 14
4. Logistic Regression- 3



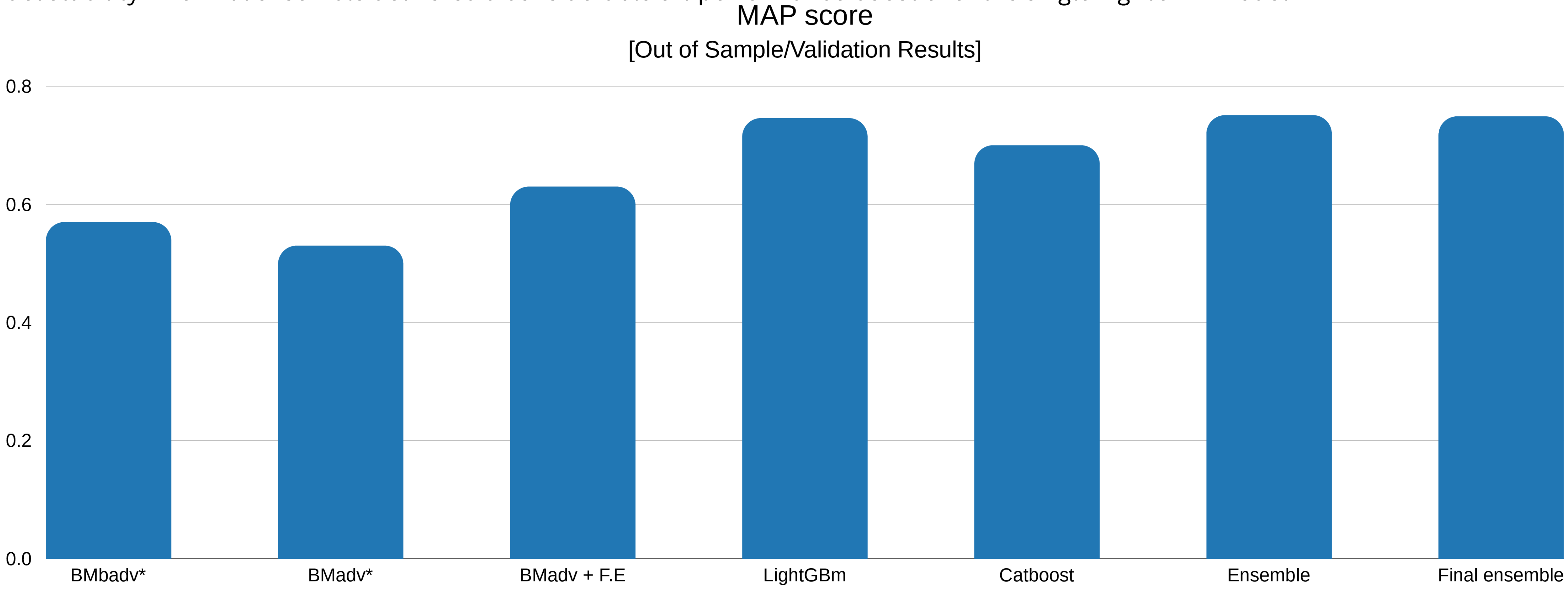
"LightGBM dominates the ensemble; XGBoost adds minor lift; CatBoost contribution is negligible."

"Final stack = LightGBM + XGBoost (CatBoost removed after ablation)"

Reference Paper:- https://doi.org/10.55524/ijirem.2025.12.1.4

Model 1 Performance – Stacked Ensemble

Our stacked ensemble's success was driven by the strong standalone performance of LightGBM, which served as our best-performing base model. Through an ablation study, we determined that CatBoost contributed more noise than signal; therefore, we removed it from the stack. While this caused a marginal decrease in the validation score, it improved model stability. The final ensemble delivered a considerable 5% performance boost over the single LightGBM model.



Graph Analysis

- Although adversarial selection reduced the initial validation score, it was essential for removing unstable features and ensuring robust performance on unseen data with better generalization.
- This performance dip was then overcome by strategic feature engineering, which significantly boosted the base model's score.
- LightGBM was the strongest standalone model and the core of our final stack.
- The final ensemble delivered the best results, outperforming the single LightGBM model while remaining efficient by removing the less impactful CatBoost model.

*BMbadv - B.M before Adversal selection , BMadv - B.M after Adversal Selection , Ensemble *= B.M + CatBoost + LightGBm , Final Ensemble* = B.M + LightGBM

*Stacked Ensemble via logistic regression

*Output from a out of sample/validation data , trained on day 0 and day 1 and validated on day 2

More Potential to Improve

Feature Engineering:

.Construct a user-offer interaction graph and use graph embedding techniques (like Node2Vec) to create features that capture latent, non-obvious relationships between users and offers.

Generate more powerful features by creating explicit correlations across all data domains like transaction, offer, events datasets.

ML Enhancements:

Explore Field-aware Factorization Machines (FFMs), a model class that excels at CTR prediction by modeling pairwise feature interactions in a more granular way.

Cluster users into behavioral segments (e.g., "High-Frequency Shoppers," "Travel Enthusiasts") and train a specialized ranking model for each to better capture their specific preferences.

Develop a separate model specifically for new offers that have no historical data. This model would be trained only on offer metadata (category, discount, brand, etc.) to predict the initial performance of a new offer more accurately than a simple global average.

Challenges:

The dataset was highly imbalanced, which was a significant problem that initially left no clear path forward for modeling.

While theoretically powerful, deep learning model DeepFM was highly sensitive to the feature drift present in the dataset and required extensive regularization and tuning, making it less robust than the final tree-based ensemble.

References :

https://www.csie.ntu.edu.tw/~cjlin/papers/ffm.pdf

https://github.com/ycjuan/libffm

https://github.com/daxiongshu/light-ffm/blob/master/light-ffm.pdf